

CUDA 架構下的平行化局部診斷設計與模擬^{*}

鄭偉仁

國立東華大學資訊工程系
610021069@ems.ndhu.edu.tw

賴寶蓮

國立東華大學資訊工程系
baolein@mail.ndhu.edu.tw

許弘駿

慈濟大學醫學資訊系
hchsu@mail.tcu.edu.tw

摘要

近年來，由於高階伺服器以及超級電腦的造價昂貴，許多大型運算紛紛採用分散式系統進行運算減少成本，但在眾多處理器所組成的網路系統裡，並無法保證所有處理器都能正常運作，隨時可能發生故障，因此，要如何能即時、正確、有效率的診斷出壞掉的處理器，是一項重要的議題。局部診斷演算法，可診斷任意一點或一小部分的點 (*processors*)，但若對一個大型網路結構做全面性診斷，則須診斷的點數和輸入資料量都會非常龐大。*CUDA* (*Compute Unified Device Architecture*) 是 *NVIDIA* 平行運算架構，因為它利用大量的 *GPU* 核心來彌補 *CPU* 核心不足大幅提升運算效能的特性，*CUDA* 近年來在各個科學研究領域中大受青睞，尤其在龐大的運算上越能看出運用平行運算的效果。因此本論文基於局部診斷演算法，設計了在 *CUDA* 平台上的症狀平行模擬程式，以平衡超立方體為診斷對象，我們提供在 *CUDA* 平台上模擬診斷的實驗數據並分析其結果，呈現局部診斷演算法的平行效率。

1 簡介

由於高階伺服器及超級電腦的造價昂貴，許多大型運算紛紛採用分散式系統進行運算降低成本，但藉助網路所實現大規模集群 (*cluster*) 或是分散式並行的架構系統裡，並無法保證所有電腦都能正常運行，隨時可能有電腦故障，因此如何能即時、正確、有效率的診斷出壞掉的電腦，是一項重要議題。

影響分散式系統效能的因素，除了硬體之外，網路結構亦相對重要；在分散式系統中一台電腦可看作一個節點，節點之間藉由網路進行傳輸與溝通，而選擇具有良好特性的圖形結構作為分散式系統的基底，可使資料傳輸 (*datatransmission*)、廣播 (*broadcasting*)、路由選擇 (*routing*) 的成本降低，在進行壞點 (*faultynode*) 診斷時，可以簡單的規則及較好的效率找出壞點，而超立方體和以其為基礎所改良的雙扭立方體、交錯立方體、平衡超立方體等，皆為較常被選用的分散式系統網路架構。

t-局部可診斷 (*localt-diagnosable*) 的概念 [4] 指的是圖中單一節點的可診斷度，以此概念所設計的局部診斷演算法縮小了診斷範圍，能夠針對單一節點或者特定區域的點做診斷，可將每個點的可診斷力各自發揮到極限，診斷力高的點不再受診斷力差的點拘束。

執行診斷演算法的平台可以是單一電腦或者分散式系統，亦能運用平行運算平台來處理；以效率來看，單一電腦需獨力診斷系統中全部節點，運算與資料皆無法分攤，而分散式系統可將診斷工作平均分配給各節點，再把所有節點的診斷結果回收到一台電腦之中，其缺點是會有節點之間交換資料的時間浪費，且傳輸資料受網路速度限制，單一電腦沒有這個問題，儘管如此，當診斷規模龐大時，單一電腦跟分散式的處理速度將清晰可辨。

CUDA 是平行運算平台之一，它利用協同處理的概念讓中央處理器 (*CPU*) 和顯示卡圖形處理器 (*GPU*) 能互相協調，共同參與運算，雖然 *CPU* 核心運算的計算效能強於 *GPU*，但是在數量以及造價上 *CPU* 核心並無法和 *GPU* 共同比擬，因此選用 *GPU* 做平行運算會是一種平價且高效能的運算方式，目前 *CUDA* 技術已被廣泛地運用在各個科學領域，諸如生物資訊、醫學、地震探勘.. 等，並取得豐碩的成果 [2] [6] [8] [11] [18]; *CUDA* 因為其運作上只需備有 *NVIDIA GeForce 8* 系列之後的顯卡，即可讓程式開發者透過 *C* 語言，設計資料分配 (*data decomposition*) 及程式流程將運算工作分配到上千個執行緒 (*threads*) 及圖形處理器中數以百計的計算核心 (*cores*) 進行運算。目前常見的三千元 *NVIDIA* 顯示卡，就能達到 *GPGPU* 平價的高效能運算方式，因此相較於其他平行平台，*CUDA* 在研究與開發上會相對容易。

局部診斷演算法在不同的網路架構上也會因圖型規則的不同而有不同的效率，先前已有相關的局部診斷演算法模擬，以超立方體和以其為基礎改良的雙扭立方體、交錯立方體為診斷對象的相關研究 [16] [17]；本論文則致力於局部診斷演算法於 *CUDA* 平台上的模擬，以平衡超立方體為診斷對象，記錄診斷時間與結果，並為模擬程式作分析。

2 局部診斷

一個圖形若為 *t*-可診斷 (*t-diagnosable*) [3]，則在壞點數量小於或等於 *t* ($\leq t$) 的情況下都能正確

^{*}本研究接受國科會編號：NSC 101-2115-M-259-007 以及 NSC 101-2221-E-320-007 研究計畫經費補助

診斷出所有壞點，因此若要針對某個圖形結構設計診斷演算法，需先得知該圖形的整體可診斷度 (*diagnosability*) t 是多少之後，才能設計演算法並判斷在各種情境下可否正確診斷，目前大部分的演算法會一次測試全圖以確認所有點的好壞，但無法單就圖中某個區塊或某個點進行診斷；另外，假設某一圖形 G 可診斷度是 t ，則當壞點數量是 $t+1$ 時無法正確找出所有壞點，但不代表 G 中所有點都無法確認好壞，假設將 G 切割成 A 、 B 兩個區塊後， A 區塊的可診斷度是 $t+1$ 、 B 區塊是 t ， A 區塊其實能容忍更高的壞點數，然而以圖 G 的整體可診斷度來看則會忽視 A 區塊較佳的可診斷度，因此當 A 和 B 兩區塊的可診斷度差距很大時，為求圖形的整體可診斷度而忽略 A 區各點的可診斷度，將造成 A 區可診斷力的浪費。

平衡超立方體 (*Balanced Hypercube*) 是由 Huang 和 Wu 在 1995 年所定義的超立方體的變形 [9] [10]，是至今已被廣泛研究的一種圖形 [12] [13] [14]，它擁有良好的結構與特性，例如對稱性 (*symmetry*)、連通性 (*connectivity*)，並且平衡超立方體在奇數維度有較超立方體低的直徑 (*diameter*)，並且在平衡超立方體裡，每個處理器都有備份的處理器，並且共享相同的一組相鄰節點，因此會被作為分散式系統的網路架構，其它相關的立方體也已有許多診斷的研究 [1] [5] [15]；我們把 n 維的平衡超立方體記為 BH_n ，它擁有 4^n 個頂點、每個點有 $2n$ 條邊、連通度 $2n$ 、可診斷度 (*diagnosability*) $2n$ 、直徑在偶數維度或是維度為 1 時皆為 $2n$ ，在奇數維度則是 $2n-1$ ，每個節點都擁有 n 個位元所組成的獨一無二的座標，分支度皆為 $2n$ ；連線規則也相當單純，偶數座標點連線至下一區的奇數座標點，定義 1 為平衡超立方體的完整連線規則，圖 1 為 BH_1 的圖形結構，圖 2 為 BH_2 的圖形結構。

定義 1. [9] 一個 n 維的平衡超立方體 BH_n 它擁有 4^n 個點 $(a_0, a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1})$ 每個 a_0 和 $a_i \in \{0, 1, 2, 3\}$ ($1 \leq i \leq n-1$) 每個點 $(a_0, a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1})$ 依照下列規則連接 $2n$ 個點

1. 內部點連線規則

$$((a_0 + 1) \bmod 4, a_1, \dots, a_{n-1}), \\ ((a_0 - 1) \bmod 4, a_1, \dots, a_{n-1}), \text{and}$$

2. 外部點連線規則

$$((a_0 + 1) \bmod 4, (a_i + (-1)^{a_0}) \bmod 4, \dots, a_{n-1}), \\ ((a_0 - 1) \bmod 4, (a_i + (-1)^{a_0}) \bmod 4, \dots, a_{n-1}),$$

當進行壞點診斷時，須先確立測試者 (*tester*) 和受測者 (*testee*)，它們各代表一個節點，且兩點必須要直接連線；進行測試時，測試者傳送信息給受測者，受測者接收信息後，回傳信息給測試者，測試者再根據傳回的信息，判斷受測者的狀態，決定測試結果；實際測試方式與使用的診斷模

型有關，目前已有許多網路診斷模型，較常被應用的是 PMC 模型 (*PMCMModel*) [7]。

假設 u, v 是圖 $G(V, E)$ 上的相鄰兩點，我們讓 u 作為測試者、 v 是受測者， u 測試 v 得到的測試結果記為 $\sigma(u, v)$ ，PMC 模型的規則如下，若 u 和 v 都是好點，則測試結果 $\sigma(u, v) = 0$ ，若 u 是好點且 v 是壞點，則 $\sigma(u, v) = 1$ ，當測試者是壞點時則不論受測者好壞，測試結果可能是 $\sigma(u, v) = 0$ 或 $\sigma(u, v) = 1$ ，測試規則如表 1，經過測試得到的結果我們稱為「症狀」(*syndrome*)。

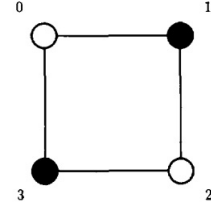


圖 1: BH_1 的圖形結構

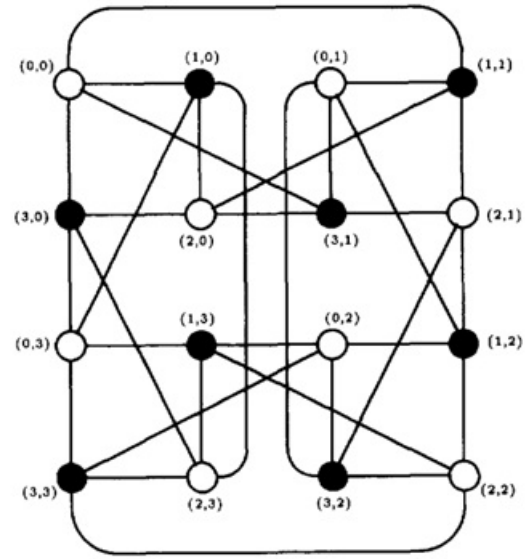


圖 2: BH_2 的圖形結構

測試者	受測者	測試結果
好	好	0
好	壞	1
壞	好	0 或 1
壞	壞	0 或 1

表 1: PMC 模型的測試規則

局部診斷演算法在局部診斷圖中的一點時，會先計算該點的局部可診斷度，Hsu 等人 [3] 提出一個稱作 T_1 的結構，我們稱它為「延展星」(extended star) 結構，符號標記為 S ，在使用 PMC 模型的情況下，它讓局部診斷演算法能確立圖形中任意一點的局部可診斷度，延展星的結構定義以及定理如下所示。

定義 2. [3] $G(V, E)$ 是一個圖形，設 v 是 G 中任意一點， k 是一個正整數，且 $k \geq 1$ ，以 v 為根點， k 條分支的延展星結構 $S(v; k)$ 定義如下：

$$S(v; k) = |V(v; k), E(v; k)|$$

$S(v; k)$ 擁有 $2k + 1$ 個點和 $2k$ 條邊，其點與邊的定義如下：

1. $V(v; k) = \{v\} \cup \{x_i, y_i | 1 \leq i \leq k\}$
2. $E(v; k) = \{(v, x_i), (x_i, y_i) | 1 \leq i \leq k\}$

圖 3 是以 v 為根點所長出的延展星結構範例。

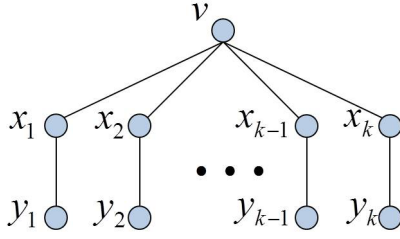


圖 3: $T_1(v; k)$ 的結構

定理 1. [3] $G(V, E)$ 是一個圖形，設 v 是 G 中任意一點，若 G 中找得到以 v 為根點、 t 條分支的子圖 $S(v; t)$ ，則 v 是 t -局部可診斷。

由上述內容，我們得知局部診斷演算法如何決定一點的局部可診斷度，從圖 3 可看出，為點 v 建立延展星的重點是替點 v 的所有鄰點找到匹配 (matching)，接著我們將介紹匹配的方法。

在 2010 年 Kung 等人 [4] 提出隨機壞點診斷演算法 (Algorithm Diagnose Random Fault)，它說明如何在 PMC 模型之下根據測試得到的症狀以及延展星結構來判斷點的好壞，此演算法的流程，假設圖形 G 有一點 v ， v 的分支度是 k 並已在 G 上找到一個 $S(v; k)$ 的結構，且 $S(v; k)$ 上的壞點數不超過 k ，設 $1 \leq i \leq k$ ，我們要依序檢驗 v 的 k 條分支；若 $(\sigma(y_i, x_i), \sigma(x_i, v)) = (0, 0)$ 代表該分支保證 v 是好點， $(\sigma(y_i, x_i), \sigma(x_i, v)) = (0, 1)$ 代表該分支保證 v 是壞點，檢驗完所有分支並作統計之後，若症狀為 $(0, 0)$ 的分支數量大於或等於 $(0, 1)$ 的分支數量，代表 v 是好點，症狀是 $(0, 0)$ 的分支數量小於 $(0, 1)$ 的分支數量，則 v 是壞點。

定理 2. [4] $G(V, E)$ 是一個圖形，設 v 是 G 中任意一點， t 是 v 的分支度，假設 G 上存在 $S(v; t)$ ，則在 $S(v; t)$ 之中壞點數量不超過 t 的情況下，隨機壞點診斷演算法能正確判斷 v 是好或壞。

3 匹配方法

從上述內容，我們運用局部診斷演算法得知一點好壞，須先替要診斷的點建構 T_1 ，建構 T_1 即是找匹配的問題，匹配的方式與圖形結構相關，因此需針對圖形設計適合的方法，才能以規律且較有效率的方式找到匹配，當 T_1 建構完成之後，即以 Kung 等人所提出的隨機壞點診斷演算法判斷點的好壞，若要診斷整個圖形，則對每個點執行局部診斷演算法，即可完成全圖診斷。

我們提供一種平衡超立方體所能使用的匹配方法。假設 n 是維度， i 是圖形中一點的編號且 $0 \leq i \leq 2n - 1$ ，本匹配方法可為任意一點 a 建構 $T_1(i; n)$ ，時間複雜度為 $O(n)$ ，下述是方法流程，而圖 4 是使用本方法建構 T_1 的例子。

Algorithm 1 匹配演算法

輸入: 圖中任意一點 a vertex($a_0, a_1 \dots a_{n-1}$)

輸出: a vertex 的 T_1 結構

```

1: if  $a_0$  is even then;
2:    $x_1 = ((a_0 + 1) \bmod 4, a_1, \dots, a_{n-1})$ ;
3:    $y_1 = ((a_0 + 1) \bmod 4, (a_1 - 1) \bmod 4, \dots, a_{n-1})$ ;
4:    $x_2 = ((a_0 - 1) \bmod 4, a_1, \dots, a_{n-1})$ ;
5:    $y_2 = ((a_0 + 1) \bmod 4, (a_1 - 1) \bmod 4, \dots, a_{n-1})$ ;
6:   for  $i = 1; i < n; i++$  do
7:      $x_{i-1} = ((a_0 + 1) \bmod 4,$ 
8:                $(a_i + 1) \bmod 4, \dots, a_{n-1})$ ;
9:      $y_{i-1} = ((a_0 + 1) \bmod 4, (a_i), \dots, a_{n-1})$ ;
10:     $x_i = ((a_0 - 1) \bmod 4,$ 
11:            $(a_i + 1) \bmod 4, \dots, a_{n-1})$ ;
12:     $y_i = ((a_0 + 1) \bmod 4, (a_i), \dots, a_{n-1})$ ;
13:   end for
14: end if
15: if  $a_0$  is odd then;
16:    $x_1 = ((a_0 + 1) \bmod 4, a_1, \dots, a_{n-1})$ ;
17:    $y_1 = ((a_0 - 1) \bmod 4, (a_1 + 1) \bmod 4, \dots, a_{n-1})$ ;
18:    $x_2 = ((a_0 - 1) \bmod 4, a_1, \dots, a_{n-1})$ ;
19:    $y_2 = ((a_0 - 1) \bmod 4, (a_1 + 1) \bmod 4, \dots, a_{n-1})$ ;
20:   for  $i = 1; i < n; i++$  do
21:      $x_{i-1} = ((a_0 + 1) \bmod 4,$ 
22:                $(a_i - 1) \bmod 4, \dots, a_{n-1})$ ;
23:      $y_{i-1} = ((a_0 + 1) \bmod 4, (a_i), \dots, a_{n-1})$ ;
24:      $x_i = ((a_0 - 1) \bmod 4,$ 
25:             $(a_i - 1) \bmod 4, \dots, a_{n-1})$ ;
26:      $y_i = ((a_0 + 1) \bmod 4, (a_i), \dots, a_{n-1})$ ;
27:   end for
28: end if

```

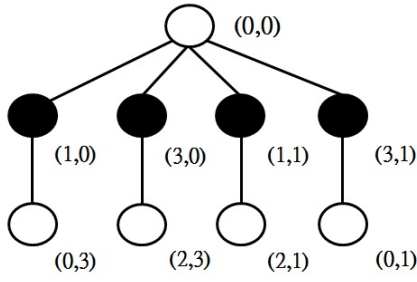


圖 4: 以匹配演算法，替 BH_2 上的點 $(0,0)$ 建構的 T_1

4 CUDA 架構下局部診斷演算法

在執行局部診斷演算法時，必須知道圖形的結構與測試後得到的症狀，這裡將圖形結構和症狀各自存放到兩個檔案（分別稱為檔案 1 和檔案 2），並將檔案 1 和檔案 2 的讀取路徑以及圖形總點數、圖形最大分支度、區塊數量、區塊包含的執行緒數量作為程式的輸入，完成這些前置動作即可開始執行程式，首先被執行的運算環境是主機的部分，其工作如下所示。

Algorithm 2 Host

輸入：由檔案 1, 檔案 2 二檔案讀取路徑、圖形總點數 $number$ 、圖形最大分支度 $MaxDegree$ 、設定區塊數量 $blocks$ 、設定區塊包含的執行緒數量 $threads$

輸出：所有的壞點編號

```

1: 讀取檔案 1, 全部圖形結構內容存入
2:   link[number * MaxDegree];
3: 讀取檔案 2, 全部症狀內容存入
4:   syndrome[number * MaxDegree];
5: cudaMalloc(CudaLink,
6:   number * MaxDegree * sizeof(int));
7: cudaMalloc(CudaSyndrome,
8:   number * MaxDegree * sizeof(int));
9: cudaMalloc(CudaResult,
10:  number * sizeof(int));
11: cudaMemcpy(CudaLink, link,
12:  number * MaxDegree * sizeof(int),
13:  cudaMemcpyHostToDevice);
14: cudaMemcpy(CudaSyndrome, syndrome,
15:  number * MaxDegree * sizeof(int),
16:  cudaMemcpyHostToDevice);
17: Diagnosis<<< blocks, threads >>>
18:   (CudaLink, CudaSyndrome,
19:   number, MaxDegree); //核心函式
20: cudaMemcpy(FaultyNodes, CudaResult,
21:  number * sizeof(int),
22:  cudaMemcpyDeviceToHost);
23: 輸出 FaultyNodes;
24: 結束

```

步驟 1 ~ 4 讀取症狀與連線規則的檔案，步驟 5 ~ 8 是為了將症狀與連線規則的資料存入裝置，事先配置裝置記憶體空間，步驟 9 ~ 10 是在裝置配置存放診斷結果的記憶體空間，步驟 11 ~ 16 則是將連線規則與症狀複製到裝置記憶體中，步驟 17 ~ 19 是傳參數呼叫核心函式，此函式為在裝置執行的診斷運算，待所有的執行緒都執行完畢之後，步驟 20 ~ 23 由主機從裝置將診斷結果複製回主機記憶體中，並在輸出診斷結果後結束工作，在裝置上每個執行緒的運算工作如下。

Algorithm 3 Diagnosis Kernel

輸入：圖形總點數、圖形最大分支度、圖形結構、圖形症狀

輸出：診斷範圍內的壞點編號

```

1: 計算負責的點範圍，得到起點編號 start、終點編號 end;
2: for i = start to end do
3:   匹配演算法(i);
4:   counter1 = 症狀為 (0,0) 的分支數量;
5:   counter2 = 症狀為 (0,1) 的分支數量;
6:   if counter1 < counter2 then
7:     i 是壞點，存入 CudaResult;
8:   end if
9: end for
10: 結束

```

步驟 1 根據自己的編號算出負責的範圍，執行緒彼此負責的點數數量差距不超過 1，步驟 2 ~ 5 執行診斷工作，若判斷是壞點，則存入在主機端事先配置好的共用記憶體空間，當負責的點診斷完畢時即結束工作，圖 5 是本方法的流程圖。

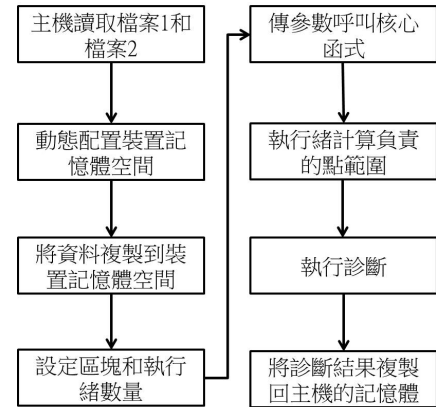


圖 5: 利用局部診斷演算法來診斷全圖的 CUDA 程式運作流程

5 實驗結果

本次實驗的 CUDA 環境，硬體設施為 CPU i7 970 3.2GHz、記憶體 4G、硬碟 500GB、作業系統為 Windows 7 32 位元，CUDA 版本為 5.0，顯示卡 Nvidia GTX 460 v2。在此環境上，受限系統記憶體大小的緣故，本實驗的診斷上限是到 10 維 1,048,576 個點，我們首先以局部診斷演算法來診斷各種維度的平衡超立方體，並分析它們的運算時間，接著是針對設定不同區塊 (blocks) 執行緒 (threads) 的組合，探討在 CUDA 實驗上的最佳化，及分析診斷結果的數據。

維度	點數	時間(秒)
2	16	0.001
3	64	0.001
4	256	0.001
5	1024	0.002
6	4096	0.002
7	16384	0.003
8	65536	0.016
9	262144	0.046
10	1048576	0.192

表 2: 以局部診斷演算法，診斷各種維度的平衡超立方體所耗費的時間

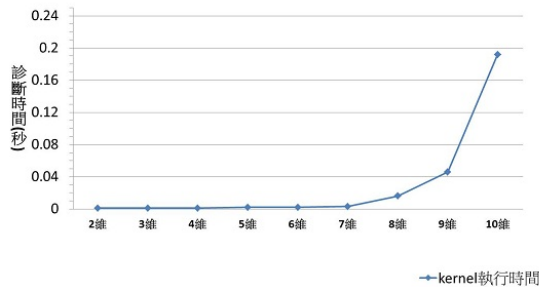


圖 6: 以局部診斷演算法，診斷各種維度的平衡超立方體所耗費的時間

觀察表 2、圖 6，可發現在維度小於 8 時，診斷都在相當快速且接近的時間完成，直到診斷點數大於 65,536 時，診斷速率才開始有明顯攀升，但即使是點數過百萬點，診斷的效率依舊在 1 秒內即可完成。接著我們再以收集到大量的實驗數據，針對 CUDA 程式區塊 (block) 執行緒 (threads) 的設定來做分析。由圖 7 開始，我們開始針對不同的計算量點數的多寡，給予不同的執行緒，並統計其產生的數據。可以看到，設定越多的執行緒對於計算效率的提升是有明顯的幫助，但是在分配的執行緒多於總計算點數的一倍時，將測試不到結果，這是因為有過多的執行緒處於沒有工作的狀態，因此 CUDA kernel 會啟動不完全；圖 8 的結果很有意思，圖中四條曲線，分別表示平衡超立方體在維度 7 到 9，設定不同的執行緒來做局部診斷計算的時間圖，我們可以理解執行緒越少會造成運算時間越長，但是我們的實驗數據也顯示執行緒越多不一定

造成運算時間的降低。我們可以發現大約在一個執行緒負責 4 個點運算時會有最佳的結果，這是因為 CUDA 運算模式的關係，若是執行緒數量，乘上每個執行緒所需的暫存器數量，超過一個多處理器中暫存器的總數，就會使得部分資料得存放在記憶體中，就會讓 context switch 付出代價，而降低了效能。因此在圖 8 中，設定相同數量的執行緒去診斷 262,144 點、1,048,576 點的時候，並不會縮短診斷的時間反而造成時間的延長。

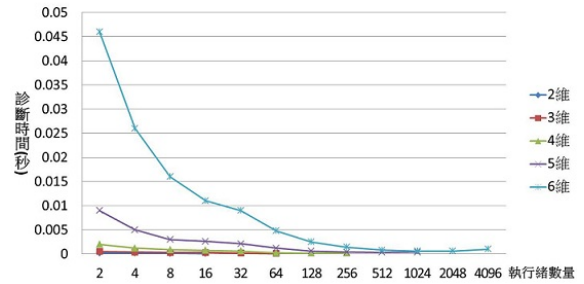


圖 7: 維度小於 6，不同執行緒診斷平衡超立方體所耗費的時間

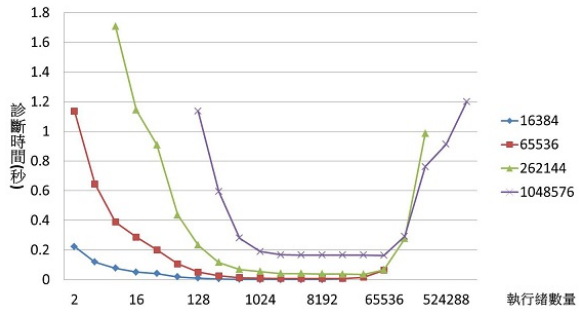


圖 8: 維度大於 7，不同執行緒診斷平衡超立方體所耗費的時間

6 結論

局部診斷演算法具有泛用性，它存在最大匹配方法，可以對圖上的每一點建構 T_1 ，因此使我們能決定該點的局部可診斷度，當該點的 T_1 壞點數不超過局部可診斷的情況下，皆可正確判斷該點狀態。

在診斷速度方面，由於 CUDA 顯示卡包含的核心數量非常多，因此能創造大量執行緒給更多核心同時執行，在平衡超立方體圖型結構，在總執行緒不超過總點數的前提下，大約一個執行緒分配 4 個點做運算會達到最佳的運算效能，當圖型點數達到 100 萬點時，診斷時間也不超過 1 秒鐘，具有即時性，未來我們計畫將局部診斷演算法套用在不同的網路圖型結構上，並且也將此實驗套用到不同的

平台上實作，例目前最流行的雲端平台 Hadoop、平行運用平台 MPI，進一步討論圖型結構不同、實作平台不同所反應出的結果。

7 參考文獻

- [1] T. Araki, "Optimal adaptive fault diagnosis of cubic Hamiltonian graphs," *Proceedings of the 7th International Symposium on Parallel Architectures 2004*, Proc. of I-SPAN 2004, Hong-Kong, May. 2004, pp. 162-167.
- [2] A. Bustamam, K. Burrage, and N. A. Hamilton, "Fast Parallel Markov Clustering in Bioinformatics using Massively Parallel Computing on GPU with CUDA and ELLPACK-R Sparse Format," *IEEE/ACM Transactions on Computational Biology and Bioinformatics 99 (PrePrints) IEEE Computer Society*, Los Alamitos, CA, USA, 2011.
- [3] G. H. Hus and J. J. M. Tan, "A local diagnosability measure for multiprocessor systems," *IEEE Trans. Parallel and Distributed Systems*, vol. 18, no. 5, pp. 598-607, 2007.
- [4] T. L. Kung, H. C. Chen, and J. J. M. Tan, "On the faulty sensor identification algorithm of wireless sensor networks under the PMC diagnosis model," *IEEE Trans. Networked Computing and Advanced Information Management, NCM*, vol. I, Seoul, South Korea, Aug. 2010, pp. 657-661.
- [5] P. C. Li, Z. D. Liu, and P. L. Lai, "Three-round adaptive diagnosis in crossed cubes," *Proceeding of the 28th Workshop on Combinatorial Mathematics and Computation Theory*, Penghu, Taiwan, May 2011, pp. 254-259.
- [6] C. M. Liu, T. Wong, E. Wu, R. Luo, S. M. Yiu, Y. Li, B. Wang, C. Yu, X. Chu, K. Zhao, R. Li, and T. W. Lam, "SOAP3: Ultra-fast GPU-based parallel alignment tool for short reads," *Bioinformatics*, vol. 28, Issue. 6, pp. 878-879, 2012.
- [7] F. P. Preparata, G. Metze, and R. T. Chien, "On the connection assignment problem of diagnosis systems," *IEEE Trans. Electronic Computers*, vol. 16, no. 12, pp. 848-854, Dec. 1967.
- [8] B. Sukhwani, M. C. Herbordt, "GPU acceleration of a production molecular docking code," *GPGPU-2: Proceedings of 2nd Workshop on General Purpose Processing on Graphics Processing Units*, pp. 19-27, NY, USA, 2009.
- [9] J. Wu and K. Huang, "The Balanced Hypercube: A cube-based system for fault-tolerant applications," *IEEE Trans. Computers*, vol. 46, no.4, pp. 484-490, 1997.
- [10] K. Huang and J. Wu, "Area Efficient Layout of Balanced Hypercubes," *International Journal of High Speed Electronics and Systems*, vol. 6, no.4, pp. 631-646, 1995.
- [11] L. S. Yung, C. Yang, X. Wan, and W. Yu, "GBOOST: a GPU-based tool for detecting gene-gene interactions in genome-wide case control studies," *Bioinformatics*, vol. 27, Issue. 9, pp. 1,309-1,310, 2010.
- [12] M. C. Yang, "Bipanconnectivity of balanced hypercubes," *Computers and Mathematics with Applications*, vol. 60, no. 7, pp. 1,859-1,867, 2010.
- [13] M. C. Yang, "Conditional Diagnosability of Balanced Hypercubes under the PMC Model," *Information Sciences*, vol. 222, no. 10, pp. 754-760, 2013.
- [14] M. C. Yang, "Super Connectivity of Balanced Hypercubes," *Applied Mathematics and Computation*, vol. 219, Issue. 3, pp. 970-975, 2012.
- [15] X. Yang and Y.Y. Tang, "A $(4n-9)/3$ diagnosis algorithm on n -dimensional cube network," *Information Sciences*, vol. 177, no. 8, pp. 1,771-1,781, 2007.
- [16] 林泰毅, 賴寶蓮, 許弘駿, "MapReduce 架構下的平行化局部診斷設計與模擬," *Proceeding of the 29th Workshop on Combinatorial Mathematics and Computation Theory*, Taipei, Taiwan, April 2012, pp. 267-271.
- [17] 林泰毅, 賴寶蓮, 許弘駿, "MapReduce 架構下的平行化局部診斷設計與模擬," *22nd Cryptology and Information Security Conference 2012*, Taichung, Taiwan, May 2012, 第二十二屆資訊安全會議論文集光碟 (CISC 2012).
- [18] "CUDA 個案研究 GPU 應用," <http://www.nvidia.com.tw/object/tesla-case-studies-tw.html>